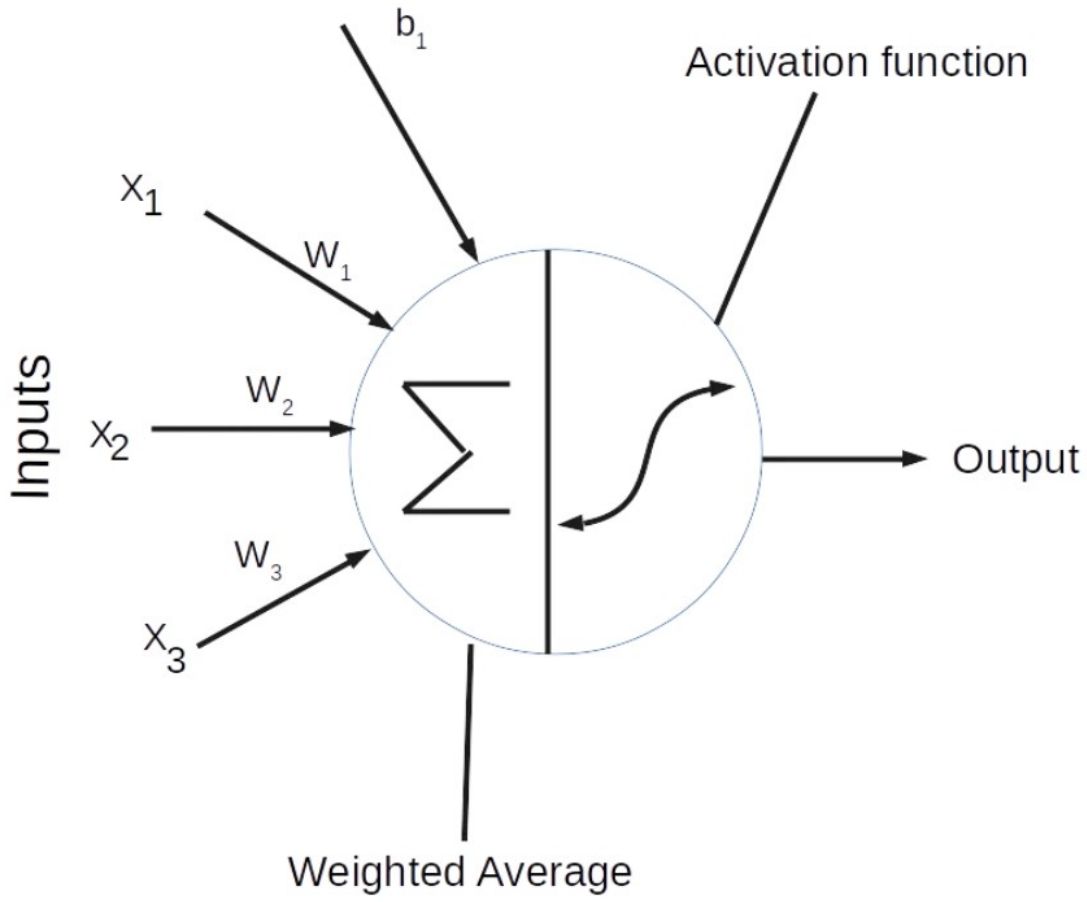
What is Deep Learning:

Deep learning is an efficient way of learning that relies on big data, where features that can help a machine map an input to an output is automatically extracted from layers of “neurons”.

What is a Neural Network:

Neural networks are composed of simple building blocks called neurons. It's a mathematical function that takes data as input, performs a transformation on them, and produces an output.



Looking at the neuron above, you can see that it’s composed of two main parts: the summation and the activation function. A neuron takes data (x_1, x_2, x_3) as input, multiplies each with a specific weight (w_1, w_2, w_3), and then passes the result to a nonlinear function called the activation function to produce an output.

A neural network combines multiple neurons by stacking them vertically/horizontally to create a network of neurons, hence the name “neural network”. A simple one-neuron network is called a perceptron and is the simplest network ever.

We need to keep in mind the big picture here:

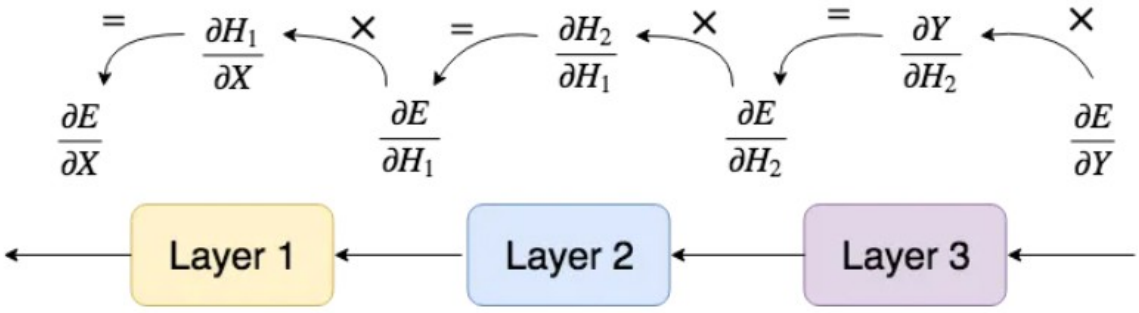
1- We feed input data into the neural network. 2- The data flows from layer to layer until we have the output. 3- Once we have the output, we can calculate the error which is a scalar. 4- Finally we can adjust a given parameter (weight or bias) by subtracting the derivative of the error with respect to the parameter itself. 5- We iterate through that process.

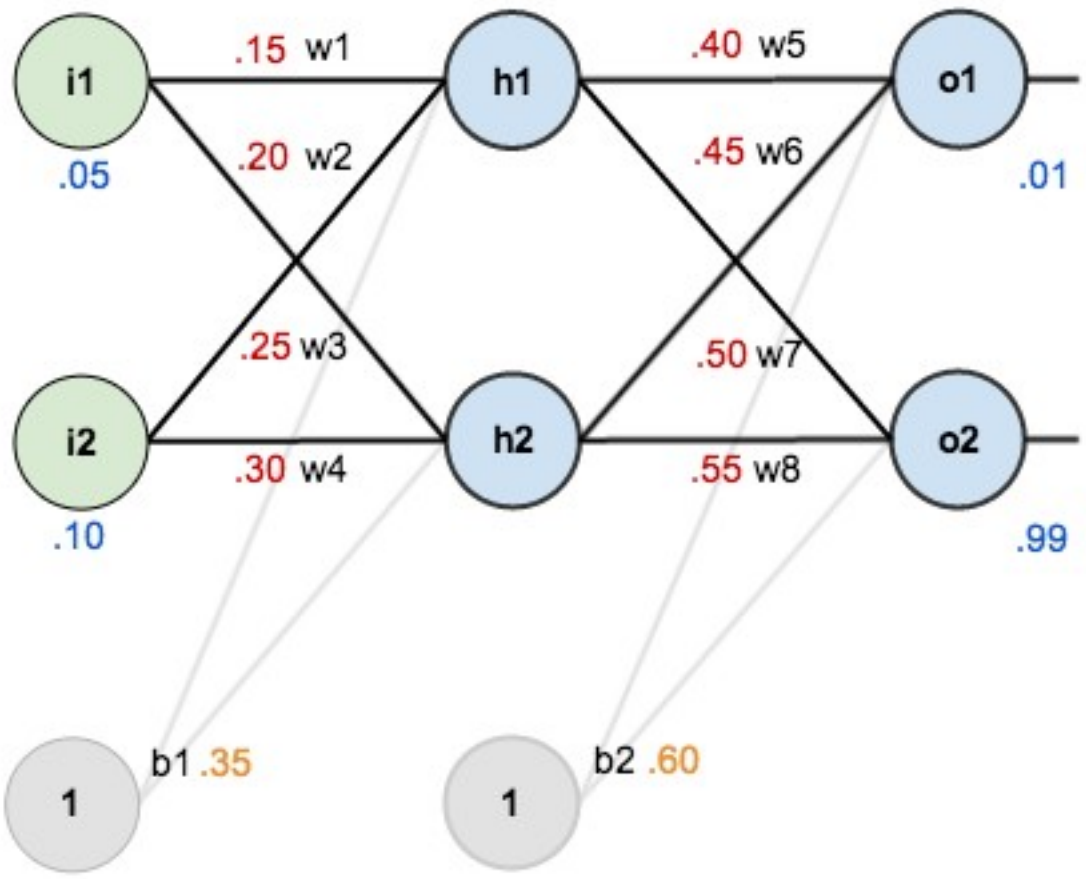
- **Forward propagation:**

We can already emphasize one important point which is: the output of one layer is the input of the next one. This is called "forward propagation".

- **Backward propagation:**

minimizing the error by changing the parameters in the network. That is "backpropagation".






```
import numpy as np

def sigmoid(x): return 1 / (1 + np.e**-x)

def sigmoid_der(y): return y * (1 - y)   # derivative when input is
already sigmoid output

# Inputs
x1, x2 = 0.05, 0.10
# Targets
t1, t2 = 0.01, 0.99
# Initial weights
w1, w2, w3, w4 = 0.15, 0.20, 0.25, 0.30 # input layer
w5, w6, w7, w8 = 0.40, 0.45, 0.50, 0.55 # hidden layer
# Biases
b1, b2 = 0.35, 0.60
# Learning rate
eta = 0.5
```

```
# Forward pass
h1_in = x1 * w1 + x2 * w2 + b1
h2_in = x1 * w3 + x2 * w4 + b1
h1_out = sigmoid(h1_in)
h2_out = sigmoid(h2_in)

o1_in = h1_out * w5 + h2_out * w6 + b2
o2_in = h1_out * w7 + h2_out * w8 + b2
o1_out = sigmoid(o1_in)
o2_out = sigmoid(o2_in)

## Errors
E1 = 0.5 * (t1 - o1_out) ** 2
E2 = 0.5 * (t2 - o2_out) ** 2
E_total = E1 + E2

# Backward pass
# Output layer
dE_outo1 = - (t1 - o1_out)
dE_outo2 = - (t2 - o2_out)
dOuto1_o1 = sigmoid_der(o1_out)
dOuto2_o2 = sigmoid_der(o2_out)

dE_dw5 = dE_outo1 * dOuto1_o1 * h1_out
dE_dw6 = dE_outo1 * dOuto1_o1 * h2_out
dE_dw7 = dE_outo2 * dOuto2_o2 * h1_out
dE_dw8 = dE_outo2 * dOuto2_o2 * h2_out
dE_db2 = dE_outo1 * dOuto1_o1 + dE_outo2 * dOuto2_o2   # bias gradient
at output layer

# Backward pass
# Hidden layer
dEout1_outh1 = dE_outo1 * dOuto1_o1 * w5
dEout2_outh1 = dE_outo2 * dOuto2_o2 * w7
dE_dOuth1 = dEout1_outh1 + dEout2_outh1
dOuth1_h1 = sigmoid_der(h1_out)
dE_h1 = dOuth1_h1 * dE_dOuth1

dEout1_outh2 = dE_outo1 * dOuto1_o1 * w6
dEout2_outh2 = dE_outo2 * dOuto2_o2 * w8
dE_dOuth2 = dEout1_outh2 + dEout2_outh2
dOuth2_h2 = sigmoid_der(h2_out)
dE_h2 = dOuth2_h2 * dE_dOuth2

dE_dw1 = dE_h1 * x1
dE_dw2 = dE_h2 * x2
dE_dw3 = dE_h1 * x1
dE_dw4 = dE_h2 * x2
dE_db1 = dE_h1 + dE_h2   # bias gradient at hidden layer
```

```
# Update weights and biases
w1 -= eta * dE_dw1
w2 -= eta * dE_dw2
w3 -= eta * dE_dw3
w4 -= eta * dE_dw4
w5 -= eta * dE_dw5
w6 -= eta * dE_dw6
w7 -= eta * dE_dw7
w8 -= eta * dE_dw8
b1 -= eta * dE_db1
b2 -= eta * dE_db2

print("Outputs:", round(o1_out, 4), round(o2_out, 4))
print("Initial Error:", round(E_total, 4))
print("\nUpdated Weights and Biases:")
print("w1 =", round(w1, 4), " w5 =", round(w5, 4))
print("w2 =", round(w2, 4), " w6 =", round(w6, 4))
print("w3 =", round(w3, 4), " w7 =", round(w7, 4))
print("w4 =", round(w4, 4), " w8 =", round(w8, 4))
print("b1 =", round(b1, 4), " b2 =", round(b2, 4))
```

```
Outputs: 0.7514 0.7729
Initial Error: 0.2984

Updated Weights and Biases:
w1 = 0.1498  w5 = 0.3589
w2 = 0.1995  w6 = 0.4087
w3 = 0.2498  w7 = 0.5113
w4 = 0.2995  w8 = 0.5614
b1 = 0.3406  b2 = 0.5498
```

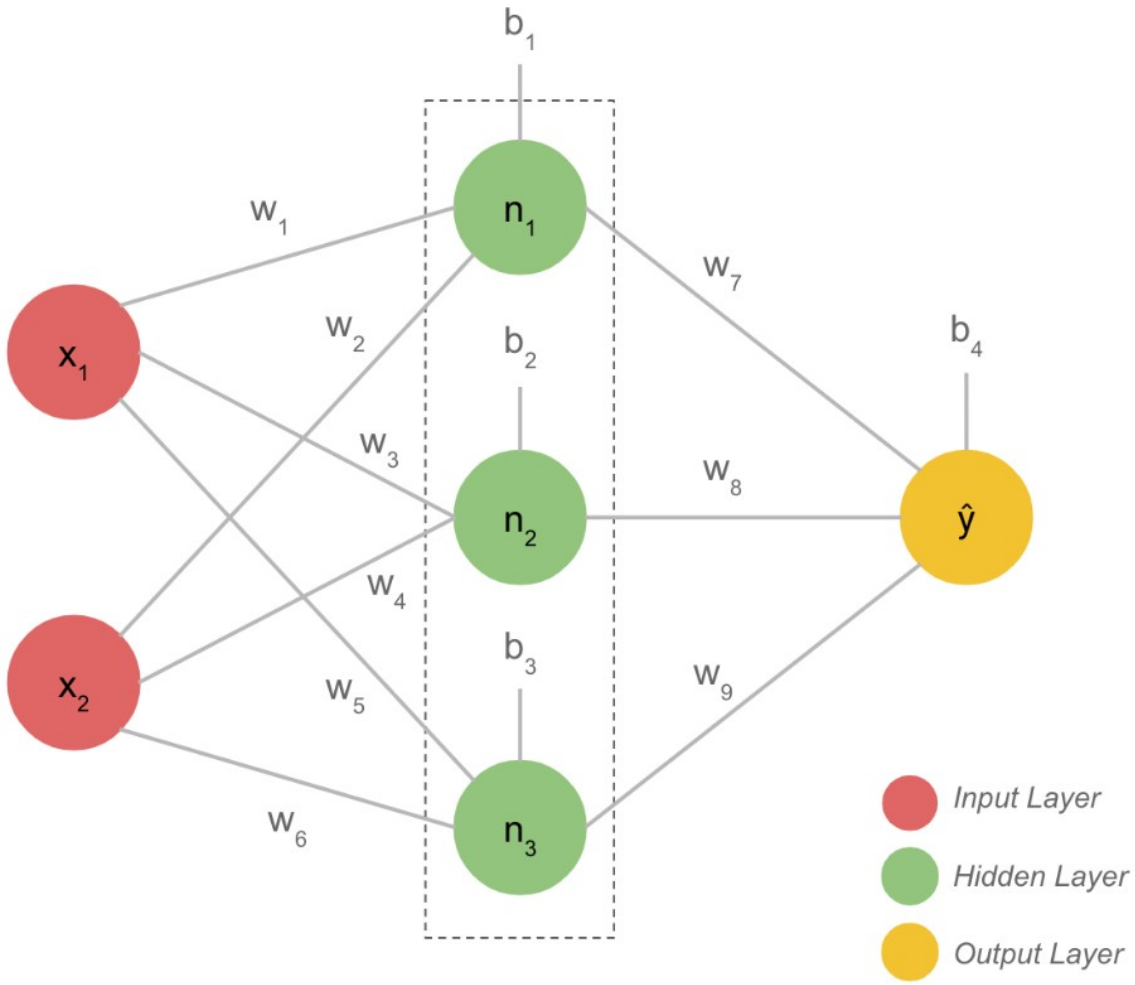
Building a Neural Network from Scratch

Import libraries

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
```

Our neural network would have three layers:

1- Input layer 2- Hidden layer with 2 neurons 3- output layer




```
housing_data = fetch_california_housing()
features = pd.DataFrame(housing_data.data,
columns=housing_data.feature_names)
target = pd.DataFrame(housing_data.target, columns=['Target'])
df = features.join(target)
df = df[['MedInc','AveRooms','Target']]
df.head()
```

	MedInc	AveRooms	Target
0	8.3252	6.984127	4.526
1	8.3014	6.238137	3.585
2	7.2574	8.288136	3.521
3	5.6431	5.817352	3.413
4	3.8462	6.281853	3.422

```
train, test = train_test_split(
    df,
    test_size=0.2,
    random_state=42
)
```

```
scaler = MinMaxScaler()
train_scaled = scaler.fit_transform(train)
test_scaled = scaler.transform(test)

X_train, y_train = train_scaled[:,:2], train_scaled[:, 2:]
X_test, y_test= test_scaled[:,:2], test_scaled[:, 2:]

X_train.shape
```

```
(16512, 2)
```

```
class SimpleNeuralNetwork:

    def __init__(self):

        # Initial weights and biases assigned random values ranging
from 0 to 1.
        # We have a total of 9 weights and 4 biases as shown in the
figure above.

        # Weights
        self.w1, \
        self.w2, \
        self.w3, \
        self.w4, \
        self.w5, \
        self.w6, \
        self.w7, \
        self.w8, \
        self.w9 = np.random.rand(9)

        # Biases
        self.b_n1, \
        self.b_n2, \
        self.b_n3, \
        self.b_y_hat = np.random.rand(4)

    # Activation functions for the neurons
    # sigmoid is for forward propagation, sigmoid derivative is for
back propagation

    def sigmoid(self, x): return 1 / (1 + np.e**-x)
    def sigmoid_der(self, x): return self.sigmoid(x) * (1 -
self.sigmoid(x))


    # Feedforward function produces result of the network prediction
for each sample.
    def feedforward(self, x):

        # x[0], x[1] - features
```

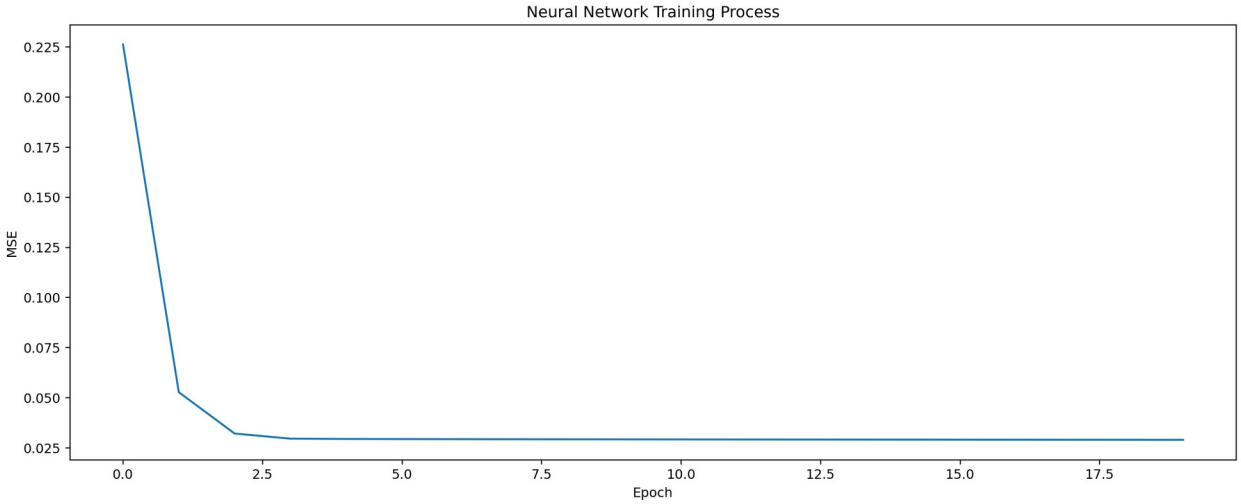
```
        # n* - neurons in the hidden layer, y_hat - predicted value
        self.n1    = self.sigmoid(x[0]*self.w1 + x[1]*self.w2 +
self.b_n1)
        self.n2    = self.sigmoid(x[0]*self.w3 + x[1]*self.w4 +
self.b_n2)
        self.n3    = self.sigmoid(x[0]*self.w5 + x[1]*self.w6 +
self.b_n3)
        self.y_hat = self.sigmoid(self.n1*self.w7 + self.n2*self.w8 +
self.n3*self.w9 + self.b_y_hat)


    # Backpropagation updates all the weights and biases of the
network.
    # By using Gradient Descent technique, each trainable parameter
(weight or bias) is changing a little
    # bit towards the minimum of MSE.
    # "output layer" => "hidden layer 2" => "hidden layer 1"

    def backpropagation(self, x, y):

        # We calculate some values here to use them later
        y_hat_der = (-(y-self.y_hat) *
self.sigmoid_der(self.n1*self.w7 + self.n2*self.w8 + self.n3*self.w9 +
self.b_y_hat))
        n1_der = self.w7 * self.sigmoid_der(x[0]*self.w1 +
x[1]*self.w2 + self.b_n1)
        n2_der = self.w8 * self.sigmoid_der(x[0]*self.w3 +
x[1]*self.w4 + self.b_n2)
        n3_der = self.w9 * self.sigmoid_der(x[0]*self.w5 +
x[1]*self.w6 + self.b_n3)

        # Biases
        self.b_n1    -= self.lr * y_hat_der * n1_der
        self.b_n2    -= self.lr * y_hat_der * n2_der
        self.b_n3    -= self.lr * y_hat_der * n3_der
        self.b_y_hat -= self.lr * y_hat_der

        # Weights
        self.w7 -= self.lr * y_hat_der * self.n1
        self.w8 -= self.lr * y_hat_der * self.n2
        self.w9 -= self.lr * y_hat_der * self.n3

        self.w1 -= self.lr * y_hat_der * n1_der * x[0]
        self.w2 -= self.lr * y_hat_der * n1_der * x[1]
        self.w3 -= self.lr * y_hat_der * n2_der * x[0]
        self.w4 -= self.lr * y_hat_der * n2_der * x[1]
        self.w5 -= self.lr * y_hat_der * n3_der * x[0]
        self.w6 -= self.lr * y_hat_der * n3_der * x[1]
```

```
    # Training process is the combination of forward and back
propagations.

    def fit(self, X, y, epoch=10, lr=0.01):

        mse_list = []
        self.lr = lr

        # Loop over epochs. Each epoch train network on all available
data.
        # We also store MSE to visualize the training process
        for i in range(epoch):
            mse = mean_squared_error(y, self.predict(X))
            mse_list.append(mse)
            print(f'Epoch: {i+1} / {epoch}, MSE: {round(mse, 4)}')

            # Loop over each training example for current epoch
            for j in range(len(X)):
                self.feedforward(X[j])
                self.backpropagation(X[j], y[j][0])

        return mse_list

    # This function is very similar to feed forward, in fact it uses
'feedforward' function to make predictions.
    # The only difference is that we predict output for all samples.

    def predict(self, X):

        result = []

        for x in X:
            self.feedforward(x)
            result.append(self.y_hat)

        return result

model = SimpleNeuralNetwork()

history = model.fit(X_train, y_train, epoch=20, lr=0.05)
```

```
Epoch: 1 / 20, MSE: 0.2267
Epoch: 2 / 20, MSE: 0.0571
Epoch: 3 / 20, MSE: 0.0539
Epoch: 4 / 20, MSE: 0.0454
Epoch: 5 / 20, MSE: 0.0344
Epoch: 6 / 20, MSE: 0.0303
Epoch: 7 / 20, MSE: 0.0294
Epoch: 8 / 20, MSE: 0.0293
Epoch: 9 / 20, MSE: 0.0292
Epoch: 10 / 20, MSE: 0.0292
```

```
Epoch: 11 / 20, MSE: 0.0292
Epoch: 12 / 20, MSE: 0.0291
Epoch: 13 / 20, MSE: 0.0291
Epoch: 14 / 20, MSE: 0.0291
Epoch: 15 / 20, MSE: 0.0291
Epoch: 16 / 20, MSE: 0.0291
Epoch: 17 / 20, MSE: 0.0291
Epoch: 18 / 20, MSE: 0.0291
Epoch: 19 / 20, MSE: 0.029
Epoch: 20 / 20, MSE: 0.029
```

```
plt.rcParams['figure.dpi'] = 227
plt.rcParams['figure.figsize'] = (16, 6)
plt.plot(history)
plt.title('Neural Network Training Process')
plt.xlabel('Epoch')
plt.ylabel('MSE')
plt.show()
```




```
y_pred = model.predict(X_test)
mean_squared_error(y_test, y_pred)
```

```
0.029724228204610608
```